```
#Enrollment : 246150307004
#Enrollment : 246150307011
#Enrollment : 246150307031

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt
```

```
# Load the dataset
df = pd.read_csv('Student_Performance.csv')
# Display first few rows and basic info
print("Dataset Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
print("\nDataset Info:")
print(df.info())
```

```
Dataset Shape: (10000, 6)

First 5 rows:
   Hours Studied  Previous Scores Extracurricular Activities  Sleep Hours  \
0              7               99                        Yes            9
1              4               82                         No            4
2              8               51                        Yes            7
3              5               52                        Yes            5
4              7               75                         No            8

   Sample Question Papers Practiced  Performance Index
0                                 1               91.0
1                                 2               65.0
2                                 2               45.0
3                                 2               36.0
4                                 5               66.0

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 6 columns):
 #   Column                            Non-Null Count  Dtype
---  ------                            --------------  -----
 0   Hours Studied                     10000 non-null  int64
 1   Previous Scores                   10000 non-null  int64
 2   Extracurricular Activities        10000 non-null  object
 3   Sleep Hours                       10000 non-null  int64
 4   Sample Question Papers Practiced  10000 non-null  int64
 5   Performance Index                 10000 non-null  float64
dtypes: float64(1), int64(4), object(1)
memory usage: 468.9+ KB
None
```

```
# Encode categorical variable
label_encoder = LabelEncoder()
df['Extracurricular Activities'] = label_encoder.fit_transform(df['Extracurricular Activities'])
print(df.head())

# Features and target
X = df.drop('Performance Index', axis=1)  # All columns except target
y = df['Performance Index']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)
```

```
   Hours Studied  Previous Scores  Extracurricular Activities  Sleep Hours  \
0              7               99                           1            9
1              4               82                           0            4
2              8               51                           1            7
3              5               52                           1            5
4              7               75                           0            8

   Sample Question Papers Practiced  Performance Index
```

```
0                               1               91.0
1                               2               65.0
2                               2               45.0
3                               2               36.0
4                               5               66.0
Training set shape: (8000, 5)
Testing set shape: (2000, 5)
```

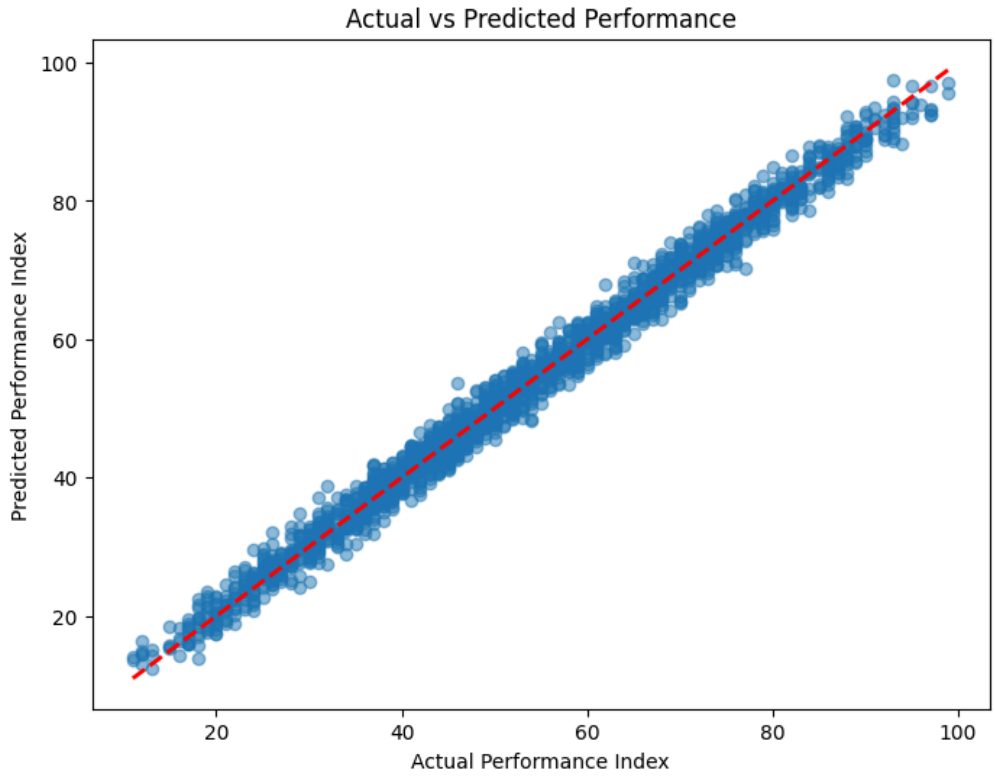
```
# Initialize and train the model
model = LinearRegression()
model.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
y_pred = model.predict(X_test)
# Evaluation metrics
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print(f"R² Score: {r2:.4f}")
print(f"RMSE: {rmse:.4f}")
```

```
R² Score: 0.9890
RMSE: 2.0206
```

```
# Scatter plot: Actual vs Predicted
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel('Actual Performance Index')
plt.ylabel('Predicted Performance Index')
plt.title('Actual vs Predicted Performance')
plt.show()
```




```
#example-1
# 7,62,YES,7,4 -> YES=1
data1 = np.array([[7,62,1,7,4]])

#example-2
#6,96,NO,9,0 -> NO=0
data2 = np.array([[6,96,0,9,0]])
pred1 = model.predict(data1)
pred2 = model.predict(data2)
```

```
print("Prediction 1:",pred1[0])
print("prediction 2:",pred2[0])
```

```
Prediction 1: 53.813242463405935
prediction 2: 85.116297820027
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feat
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feat
  warnings.warn(
```